



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LAB 17 INSTRUCTIONS

For

Compare encrypted and decrypted views

Course Code: C1123

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 5.1

Lab 17 — Compare encrypted and decrypted views

Course: Wireshark Network Analysis Masterclass (C1123)

Topic: 17 — TLS-Encrypted Traffic Analysis

Learning outcome: LO4 — Diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

Time: about 60 minutes

Version: v5.1 · 5 October 2026

Scenario

Under an authorised test, the portal's HTTPS traffic must be inspected. You compare what an analyst sees with and without the session secrets.

Goal

Without session secrets, encrypted application records do not disclose HTTP payload.

What you will produce

A comparison of the encrypted and decrypted views.

Before you start

- Wireshark 4.6 or later, with the TShark command-line tools on PATH.
- Python 3 for the verification and export scripts (Windows: use `py -3` in place of `python3`).
- Open this lab folder in a terminal; every file the lab needs is inside it.
- Use only the supplied synthetic captures — do not capture on a network you are not authorised to monitor.

Files for this lab

File	What it is for
data/tls-session.pcap	Synthetic TLS capture used by the lab
assets/scenario.md	The help-desk ticket that sets the scenario
assets/checks.json	Filters and frame counts the fixture must satisfy
assets/expected-evidence.png	The packet list your filter should produce
scripts/verify.py	Checks the capture facts with TShark
scripts/export_evidence.py	Exports this lab's evidence rows to outputs/evidence.csv
outputs/findings.md	Findings sheet you complete
data/lab-tls.keys	Synthetic session secrets for this capture only

Lab at a glance

1. Inspect the encrypted TLS view
2. Read the SNI in the ClientHello
3. Load the key log and view the HTTP
4. Verify, then remove the key log

Step-by-step

1. Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
python3 scripts/verify.py
```

2. Open `data/tls-session.pcap`. In Preferences > Protocols > TLS, clear the (Pre)-Master-Secret log filename, then apply `tls`. Identify ClientHello and encrypted application records.

3. Expand ClientHello > Extensions > `server_name`. Record `portal.example.test`. The lab uses a real TLS 1.2 MemoryBIO exchange, a temporary self-signed certificate and offline synthetic TCP wrapping.

4. Set Preferences > Protocols > TLS > (Pre)-Master-Secret log filename to the absolute path of `data/lab-tls.keys`. Reload the capture. Apply `http` and confirm GET `/health` and 200 OK containing LAB-OK.

5. Run `scripts/verify.py` for both encrypted/decrypted checks. Clear the key-log preference when finished. The supplied secrets are synthetic lab session material; never copy production session secrets into a public repository.

6. Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in `outputs/findings.md`.

```
python3 scripts/export_evidence.py
```

Expected evidence

With the lab filter applied, your packet list should match the frames below (produced by TShark from this lab's own capture).

tls.handshake.type == 1					
No.	Time	Source	Destination	Protocol	Info
4	0.060000	192.0.2.10	192.0.2.20	TLSv1	Client Hello (SNI=portal.example.test)
tls-session.pcap Displayed: 1					
http.request					
No.	Time	Source	Destination	Protocol	Info
(no packets match – expected for this view)					
tls-session.pcap Displayed: 0					
http.request [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
12	0.220000	192.0.2.10	192.0.2.20	HTTP	GET /health HTTP/1.1
tls-session.pcap Displayed: 1					
http.response.code == 200 [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
14	0.260000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK
tls-session.pcap Displayed: 1					

Lab 17 expected evidence

Test it

The supplied `tls.handshake.type == 1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Troubleshooting

- TShark not found: install Wireshark CLI tools and add the installation folder to PATH; on Windows use the Wireshark install directory.
- A filter returns zero: clear other filters, use the specified capture, and check the expression is in the display toolbar.
- TLS remains opaque: select the matching `lab-tls.keys` file by absolute path, reload, and remove a key-log preference from another lab.

Try it with TShark

The same evidence from the command line — run it from this lab folder:

```
tshark -n -r data/tls-session.pcap -o tls.keylog_file:data/lab-tls.keys  
-Y http
```

Challenge

Develop and explain an alternative filter.

Reflection

Which second observation point would strengthen your conclusion?

Extension (optional)

TLS lab — identify the handshake records and cipher suite in the TLS trace from the Kurose & Ross labs. Source: J.F. Kurose and K.W. Ross, Wireshark Labs (gaia.cs.umass.edu/kurose_ross/wireshark.php).

Reset

Clear all display filters and return to the C1123-Analyst or Default profile. If you loaded a TLS key log, remove it from Preferences > Protocols > TLS. The supplied captures never need regenerating for the core lab.

Note: The same steps appear in the Learner Guide. The slides show only the scenario and a summary.